



A Complex Engineering Problem Report on
Database Security and Privacy Engineering for
Sensitive Information Systems

A9511 - Database Management Systems

Submitted by

Batch No. 6

Madhavarapu Saritha 25881A05V7

Chepyala Vishal 25881A05X9

Gundu Srijay Krishna 25881A05X0

Course Facilitator

Dr. Reddy Saisindhutheja
Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

November 2026

Contents

1	Problem Definition and Context	2
1.1	Sensitive Data in a Student Information System	2
1.2	Why This Is a Complex Engineering Problem	3
1.3	Relevance to Sustainable Development Goals (SDGs)	3
2	Problem Analysis and Solution Development	3
2.1	Characterization of the Problem	3
2.2	Engineering Knowledge Applied	4
2.3	Threat Model	4
2.4	Roles, Least-Privilege Grants, Views and Row-Level Security	6
2.5	SQL Injection: Attack Demonstration and Fix	9
2.6	Encryption of Sensitive Columns	10
2.7	Audit Logging	12
2.8	Performance Overhead Measurement	13
2.9	Conflict Between Security, Usability and Performance	15
2.10	Proposed Solution Strategy	16
3	Sustainability Integration	16
3.1	Efficient and Responsible Data Handling	16
3.2	Impact on Society and Environment	16
4	Conclusion and Future Directions	17
4.1	Solution Summary	17
4.2	Future Scope	17

1 Problem Definition and Context

1.1 Sensitive Data in a Student Information System

A college Student Information System (SIS) holds some of the most sensitive data an institution owns. This includes identity numbers (Aadhaar), contact details and dates of birth, academic marks that decide careers, and fee-payment records. The same database is used every day by very different people: registrar staff, faculty, students, the accounts section and auditors. A single weakness, such as a vulnerable web form, an over-privileged account or an unencrypted backup, can expose the records of thousands of students at once.

Problem Statement: A student/customer database must satisfy confidentiality, integrity and data-protection regulations. Design role-based access, encryption, auditing and SQL injection defences, and analyse the conflict between security controls and usability or performance.

What has to be done:

1. Build a threat model listing assets, threats and attackers for the database.
2. Implement roles, least-privilege grants, views and row-level security.
3. Demonstrate an SQL injection on a vulnerable query and fix it with parameterized queries.
4. Encrypt sensitive columns and enable audit logging.
5. Measure the performance overhead and discuss its impact on usability.

Regulatory context. India’s *Digital Personal Data Protection (DPDP) Act, 2023* requires every data fiduciary to protect personal data by taking “reasonable security safeguards to prevent personal data breach” (Section 8(5)), and failure can attract penalties of up to Rs.250 crore. The DPDP Rules, 2025 name encryption, masking, access control and access logs as examples of such safeguards. UIDAI additionally requires Aadhaar numbers to be stored only in encrypted form in a secure “Aadhaar Data Vault”. The design in this report maps directly to these obligations.

All controls were implemented and tested on **PostgreSQL 16.14** with the **pgcrypto** extension. The application-side demonstration uses Python 3 with **psycopg2**. Every result quoted below was produced by running the scripts supplied with this report.

1.2 Why This Is a Complex Engineering Problem

Table 1: Characteristics of a Complex Engineering Problem (WP1–WP7) addressed in this work

Attribute	How it appears in this problem
WP1 Depth of knowledge	Requires DBMS internals (RLS, view security, triggers), applied cryptography (AES, HMAC, key derivation) and secure coding.
WP2 Conflicting requirements	Stronger security (encryption, auditing, masking) costs speed and convenience; the right balance has to be measured, not assumed.
WP3 Depth of analysis	Threat modelling with risk scoring, attack simulation and quantitative performance measurement.
WP4 Infrequently encountered issues	Searching encrypted data (blind index), definer- vs. invoker-rights views, and RLS policy evaluation cost.
WP5 Extent of applicable codes	DPDP Act 2023, UIDAI Aadhaar Data Vault requirement, OWASP and NIST guidance.
WP6 Stakeholder involvement	Students, faculty, registrar, accounts, auditors and regulators each need different access.
WP7 Interdependence	Roles, RLS, encryption, views and audit triggers interact; a change in one affects the others.

1.3 Relevance to Sustainable Development Goals (SDGs)

- **SDG 16 (Peace, Justice and Strong Institutions):** Protecting personal data safeguards the fundamental right to privacy (Target 16.10), and tamper-proof audit trails make institutions accountable (Target 16.6).
- **SDG 9 (Industry, Innovation and Infrastructure):** Secure-by-design databases are part of resilient and trustworthy digital infrastructure.
- **SDG 4 (Quality Education):** Students and parents can trust digital academic systems only when marks cannot be tampered with and personal data cannot leak.

2 Problem Analysis and Solution Development

2.1 Characterization of the Problem

The system stores four kinds of data with very different sensitivity (Table 2). A naive design gives the application one powerful database account and leaves every check to the application code. That design fails in three ways: one bug (for example, SQL injection) exposes everything; insiders can see far more than their job needs; and nobody can prove who changed a mark or viewed an Aadhaar number. The goal is *defence in depth*, so that the database itself enforces confidentiality and integrity even when the application layer fails.

Table 2: Data classification of the SIS

Data	Columns	Class	Main concern
Identity numbers	aadhaar_enc, phone_enc	Restricted	Confidentiality
Personal profile	full_name, email, dob	Confidential	Confidentiality
Academic records	marks.internal, marks.external	Confidential	Integrity
Financial records	fee_payment.*	Confidential	Integrity
Encryption keys	vault.data_key	Restricted	Confidentiality
Audit trail	audit.log	Internal	Integrity, non-repudiation
Reference data	department, course	Internal	Availability

2.2 Engineering Knowledge Applied

- **Security Principles:** Least privilege, separation of duties, complete mediation and defence in depth (Saltzer and Schroeder).
- **Threat Modelling:** STRIDE classification with likelihood $\times$ impact risk scoring.
- **Access Control:** Role-Based Access Control (RBAC), column-level privileges, PostgreSQL Row-Level Security (RLS), security-barrier and security-invoker views, SECURITY DEFINER functions.
- **Applied Cryptography:** AES-256 (OpenPGP symmetric encryption via pgcrypto), HMAC-SHA256 blind indexing, salted key derivation, SCRAM-SHA-256 authentication and TLS.
- **Secure Coding:** Parameterized queries and allow-list input validation against SQL injection (OWASP).
- **Performance Engineering:** Controlled benchmarking, query-plan analysis and overhead quantification.

2.3 Threat Model

2.3.1 System Context and Trust Boundaries

Figure 1 shows who talks to the database and where trust boundaries lie. Red dashed arrows are attack paths that the controls in this report must block.

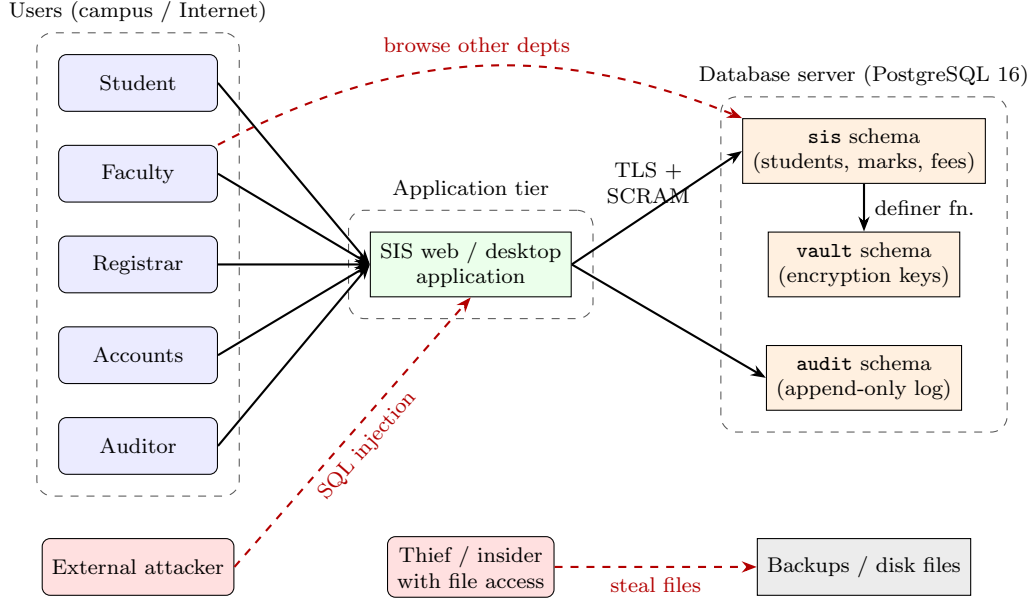


Figure 1: Data-flow diagram with trust boundaries (dashed boxes) and attack paths (red)

2.3.2 Assets and Attackers

Table 3: Threat agents (attackers)

Attacker	Motivation and capability	Typical target
External attacker	Data theft for fraud or resale; uses automated SQL injection tools	All PII, encryption keys
Curious/malicious student	Change own marks, view classmates' data; has a valid login	Marks, other students' records
Curious faculty member	Browse students of other departments; has a valid login	Student profiles
Privileged insider (registrar, DBA)	Misuse legitimate access to PII and then hide traces	Aadhaar/phone, audit log
Physical/backup thief	Copies disk images or backup files	Data at rest
Network eavesdropper	Sniffs traffic between application and database	Credentials, query results

2.3.3 STRIDE Threat Analysis and Risk Scoring

Each threat is rated for likelihood L and impact I on a 1–5 scale. The risk score is

$$R = L \times I, \quad \text{High if } R \geq 15, \quad \text{Medium if } 8 \leq R < 15, \quad \text{Low if } R < 8.$$

Table 4: STRIDE threats, risk before controls, and the control that mitigates each one

#	Threat	STRIDE	L	I	R	Control (section)
T1	SQL injection through the result portal	T, I, E	4	5	20	Parameterized queries, input validation, least privilege (2.5)
T2	Student modifies own marks	T	3	5	15	No UPDATE grant, RLS, audit (2.4, 2.7)
T3	Faculty views students of other departments	I	4	3	12	RLS + column privileges (2.4)
T4	Registrar misuses Aadhaar/phone data	I, R	3	4	12	Masking, break-glass reveal with reason and daily limit (2.6)
T5	Stolen password or brute-force login	S	3	4	12	Per-user logins, SCRAM-SHA-256, strong passwords (2.6)
T6	Theft of database files or backups	I	2	5	10	Column encryption (AES-256), keys isolated (2.6)
T7	Faculty edits marks of a course they do not teach	T	2	4	8	RLS WITH CHECK (2.4)
T8	Insider deletes audit records to hide misuse	R	2	4	8	Append-only log, separate auditor role (2.7)
T9	Eavesdropping between application and database	I	2	4	8	TLS (<code>hostssl</code>) (2.6)
T10	Expensive queries exhaust the server	D	2	3	6	<code>statement_timeout</code> , per-role connection limits

STRIDE: *S* = Spoofing, *T* = Tampering, *R* = Repudiation, *I* = Information disclosure, *D* = Denial of service, *E* = Elevation of privilege.

2.4 Roles, Least-Privilege Grants, Views and Row-Level Security

2.4.1 Role Design

Privileges are given to five *group roles* that match job functions. Each human gets a personal *login role* that inherits exactly one group role. All objects belong to `sis_owner`, a role that *cannot log in*. Default privileges for PUBLIC are revoked from the database, the schemas and every function.

Listing 1: Role hierarchy (from 01_roles.sql)

```

1 CREATE ROLE sis_owner NOLOGIN;           -- owns every object, nobody logs in as it
2 CREATE ROLE r_admin_office NOLOGIN;      -- registrar / exam cell
3 CREATE ROLE r_faculty NOLOGIN;          -- teaching staff
4 CREATE ROLE r_student NOLOGIN;          -- students (self-service)
5 CREATE ROLE r_accounts NOLOGIN;         -- fee section
6 CREATE ROLE r_auditor NOLOGIN;          -- internal audit / DPO
7
8 CREATE ROLE fac_kumar LOGIN PASSWORD '...' IN ROLE r_faculty;  -- one per person
9 CREATE ROLE stu_25881a0501 LOGIN PASSWORD '...' IN ROLE r_student;
10
11 REVOKE ALL ON DATABASE sis_db FROM PUBLIC;
12 REVOKE CREATE ON SCHEMA public FROM PUBLIC;

```

Table 5: Privilege matrix (rows visible are further limited by RLS)

Object	r_admin_office	r_faculty	r_student	r_accounts	r_auditor
student	SELECT all; UPDATE name, email, dept; INSERT only via register_ student()	SELECT 5 non-PII columns, own department	SELECT 6 columns, own row	SELECT 4 columns	–
marks	SELECT, INSERT, UPDATE	SELECT, UPDATE (internal, external), own courses	SELECT own	–	–
fee_payment	SELECT	–	SELECT own	SELECT, INSERT	–
v_student_masked	SELECT	–	–	SELECT	–
v_marks_sheet	SELECT	SELECT	SELECT	–	–
reveal_pii()	EXECUTE	–	–	–	–
audit.log	–	–	–	–	SELECT
vault.*	–	–	–	–	–

Listing 2: Column-level least-privilege grants (from 04_access_control.sql)

```

1 GRANT SELECT ON sis.student TO r_admin_office;
2 GRANT UPDATE (full_name, email, dept_id) ON sis.student TO r_admin_office;
3 GRANT SELECT (student_id, roll_no, full_name, email, dept_id) ON sis.student TO r_faculty;
4 GRANT SELECT (student_id, roll_no, full_name, email, dept_id, dob) ON sis.student TO r_student;
5 GRANT SELECT (student_id, roll_no, full_name, dept_id) ON sis.student TO r_accounts;
6
7 GRANT SELECT, INSERT, UPDATE ON sis.marks TO r_admin_office;
8 GRANT SELECT, UPDATE (internal, external) ON sis.marks TO r_faculty;
9 GRANT SELECT ON sis.marks TO r_student;
10
11 GRANT SELECT, INSERT ON sis.fee_payment TO r_accounts;
12 GRANT SELECT ON sis.fee_payment TO r_admin_office, r_student;

```

2.4.2 Row-Level Security Policies

Grants decide *which tables and columns* a role may touch; RLS decides *which rows*. Small SECURITY DEFINER helper functions map the login (`session_user`) to a faculty or student identity. Wrapping the helper as (`SELECT fn()`) makes PostgreSQL evaluate it once per query instead of once per row; Section 2.8 measures why this matters.

Listing 3: Identity helper and RLS policies

```

1 CREATE FUNCTION sis.my_dept() RETURNS SMALLINT
2 LANGUAGE sql STABLE SECURITY DEFINER SET search_path = pg_catalog, sis AS $$
3     SELECT dept_id FROM sis.faculty WHERE db_user = session_user;
4 $$;
5
6 ALTER TABLE sis.student ENABLE ROW LEVEL SECURITY;
7 CREATE POLICY student_admin ON sis.student FOR ALL TO r_admin_office
8     USING (true) WITH CHECK (true);
9 CREATE POLICY student_faculty ON sis.student FOR SELECT TO r_faculty
10     USING (dept_id = (SELECT sis.my_dept()));
11 CREATE POLICY student_self ON sis.student FOR SELECT TO r_student
12     USING (student_id = (SELECT sis.my_student_id()));
13 CREATE POLICY student_accounts ON sis.student FOR SELECT TO r_accounts
14     USING (true);
15
16 ALTER TABLE sis.marks ENABLE ROW LEVEL SECURITY;
17 CREATE POLICY marks_faculty_edit ON sis.marks FOR UPDATE TO r_faculty
18     USING (course_id IN (SELECT course_id FROM sis.course
19         WHERE faculty_id = (SELECT sis.my_faculty_id())));

```

```

20     WITH CHECK (course_id IN (SELECT course_id FROM sis.course
21                               WHERE faculty_id = (SELECT sis.my_faculty_id())));
22 CREATE POLICY marks_self ON sis.marks FOR SELECT TO r_student
23     USING (student_id = (SELECT sis.my_student_id()));
24 -- (similar policies for marks_admin, marks_faculty_read and fee_payment)

```

2.4.3 Views: Definer Rights vs. Invoker Rights

PostgreSQL views normally run with the *owner's* privileges. This is used deliberately for one view and avoided for the other:

- `v_student_masked` runs with owner rights, so it can read the ciphertext columns. It decrypts them only through `sis.mask()`, which returns just the last four characters. `security_barrier` stops a caller from using a leaky function in a `WHERE` clause to see rows before masking.
- `v_marks_sheet` is declared `security_invoker = true`, so the caller's own grants and RLS policies on `student` and `marks` decide what each user sees through the same view.

Listing 4: Masking view and invoker-rights view

```

1 CREATE FUNCTION sis.mask(p_cipher BYTEA, p_prefix TEXT) RETURNS TEXT
2 LANGUAGE sql STABLE SECURITY DEFINER SET search_path = pg_catalog AS $$
3     SELECT p_prefix || right(vault.dec(p_cipher), 4);
4 $$;
5
6 CREATE VIEW sis.v_student_masked WITH (security_barrier = true) AS
7 SELECT s.roll_no, s.full_name, d.dept_code,
8        sis.mask(s.phone_enc, 'XXXXXX') AS phone_masked,
9        sis.mask(s.aadhaar_enc, 'XXXX-XXXX-') AS aadhaar_masked
10 FROM sis.student s JOIN sis.department d ON d.dept_id = s.dept_id;
11 GRANT SELECT ON sis.v_student_masked TO r_accounts, r_admin_office;
12
13 CREATE VIEW sis.v_marks_sheet WITH (security_invoker = true) AS
14 SELECT s.roll_no, s.full_name, c.course_code,
15        m.internal, m.external, m.internal + m.external AS total
16 FROM sis.marks m
17 JOIN sis.student s ON s.student_id = m.student_id
18 JOIN sis.course c ON c.course_id = m.course_id;
19 GRANT SELECT ON sis.v_marks_sheet TO r_admin_office, r_faculty, r_student;

```

2.4.4 Verification

The script `06_verify.sql` logs in as each user (`SET SESSION AUTHORIZATION`) and tries both allowed and forbidden actions. The sample data has 6 students (3 CSE, 2 ECE, 1 MECH).

Table 6: Access-control test results

#	User	Action	Observed result	Pass
T2	fac.kumar (CSE)	SELECT ... FROM sis.student	3 rows, all CSE	✓
T3	fac.kumar	SELECT phone_enc FROM sis.student	permission denied for table student	✓
T4	fac.kumar	SELECT * FROM v_marks_sheet	6 rows, only CS301/CS302	✓
T5	fac.kumar	UPDATE a CS301 mark / an EC301 mark	UPDATE 1 / UPDATE 0 (row invisible)	✓
T6	fac.rao (ECE)	SELECT ... FROM sis.student	2 rows, both ECE	✓
T7	stu.25881a0501	Read student, marks sheet, fees	Only own row, own 2 marks, own 1 payment	✓
T8	stu.25881a0501	UPDATE sis.marks SET external = 60	permission denied for table marks	✓
T9	acc.meena	SELECT * FROM v_student_masked	XXXXXX0001, XXXX-XXXX-0001, ...	✓
T9	acc.meena	Read marks; read vault.data_key	permission denied (both)	✓
T10	admin.ravi	reveal_pii('25881A0502', 'x')	A justification of at least 10 characters is required	✓
T10	admin.ravi	DELETE FROM audit.log	permission denied for schema audit	✓
T12	sis.owner	UPDATE / TRUNCATE audit.log	audit.log is append-only (... blocked)	✓

2.5 SQL Injection: Attack Demonstration and Fix

2.5.1 The Vulnerable Query

A typical “check your result” page takes a roll number from the user and pastes it into the SQL text:

Listing 5: Vulnerable lookup (from app/injection_demo.py)

```

1 def result_lookup_vulnerable(conn, roll_no):
2     sql = ("SELECT roll_no, full_name, course_code, total "
3           "FROM sis.v_marks_sheet WHERE roll_no = '" + roll_no + "'")
4     with conn.cursor() as cur:
5         cur.execute(sql)
6         return cur.fetchall() if cur.description else []

```

Because the input becomes part of the SQL text, an attacker can close the string literal and append their own SQL. Three classic payloads were tested (Table 7).

Table 7: Injection payloads

Type	Input typed into the roll-number box	Goal
Tautology	' OR '1'='1	Make the WHERE clause always true and dump every row
UNION	' UNION SELECT purpose, encode(key_bytes,'hex'), NULL, NULL FROM vault.data_key --	Read another table: the encryption keys
Stacked query	'; UPDATE sis.marks SET external = 60; --	Run a second statement that changes data

2.5.2 Results

Part A: vulnerable code with an over-privileged account. Many applications connect as the table owner. Simulating this with `legacy_app` (a member of `sis.owner`), every attack succeeded:

- Tautology returned **all 9** marks rows of all students instead of the 2 rows of one student.
- The UNION payload returned the **encryption and HMAC keys**, which makes column encryption useless:
(‘ENC’, ‘29aec9e2c0ffbbacd06377350675...f1722c34’, None, None)
- The stacked query changed the external mark of **every** student to 60 (9 of 9 rows tampered).

Part B: the same vulnerable code with a least-privilege account. Connected as student `stu_25881a0501`, the injection still “works” syntactically, but the database limits the damage:

- Tautology returned only the student’s **own 2 rows**, because RLS filters the rows regardless of the WHERE clause.
- UNION on the keys: **permission denied** for schema `vault`.
- Stacked UPDATE: **permission denied** for table `marks`.

Part C: the fix. The input is first checked against the roll-number format, then passed as a *bound parameter*, so the driver sends it strictly as a string value and never as SQL code:

Listing 6: Fixed lookup with validation and a parameterized query

```

1 ROLL_NO = re.compile(r"[0-9]{2}881A[0-9]{2}[0-9A-Z]{2}")
2
3 def result_lookup_safe(conn, roll_no):
4     if not ROLL_NO.fullmatch(roll_no):
5         raise ValueError("invalid roll number format")
6     with conn.cursor() as cur:
7         cur.execute("SELECT roll_no, full_name, course_code, total "
8                     "FROM sis.v_marks_sheet WHERE roll_no = %s", (roll_no,))
9     return cur.fetchall()

```

Even with the over-privileged `legacy_app` account, all three payloads were rejected by validation. With validation removed, the parameterized query searched for the literal string `' OR '1'='1`, found **0 rows** and changed nothing. The SQL actually sent to the server shows that the quotes were escaped:

```
... WHERE roll_no = ''' OR ''1''=''1'
```

Lesson. Parameterized queries remove the vulnerability; least privilege and RLS limit the damage if a vulnerability is ever missed. Both layers are needed.

2.6 Encryption of Sensitive Columns

2.6.1 Design

- **Algorithm:** `pgp_sym_encrypt` (OpenPGP, RFC 4880) with **AES-256**. Each value gets a random salt and session key, so two students with the same phone number have different ciphertexts. This prevents frequency analysis, and a built-in integrity check detects tampered ciphertext.
- **Key derivation choice:** the key is 256 random bits, not a human password. Password stretching (`s2k-mode=3`) therefore adds cost but no security, and `s2k-mode=1` (salted, single pass) is used. Section 2.8 shows this makes encryption **26× faster**.
- **Key isolation:** keys live in `vault.data_key`. No role except the non-login owner has `USAGE` on the `vault` schema; keys are reachable only through `SECURITY DEFINER` functions whose `EXECUTE` right has been revoked from `PUBLIC`.

- **Searching encrypted data:** an encrypted column cannot be searched with =, because each encryption differs. A **blind index**, HMAC-SHA256(k_2 , aadhaar), stored in a **UNIQUE** column, allows exact-match lookup and duplicate detection without revealing the number.
- **Data in transit:** `ssl = on` in `postgresql.conf`, `hostssl ... scram-sha-256` lines in `pg_hba.conf`, and `password_encryption = scram-sha-256`.

Listing 7: Key vault and encryption functions (from 02.schema.sql)

```

1 CREATE TABLE vault.data_key (
2   purpose    VARCHAR(10) PRIMARY KEY CHECK (purpose IN ('ENC','HMAC')),
3   key_bytes  BYTEA      NOT NULL CHECK (length(key_bytes) = 32),
4   created_at TIMESTAMPTZ NOT NULL DEFAULT now()
5 );
6 INSERT INTO vault.data_key (purpose, key_bytes) VALUES
7   ('ENC', gen_random_bytes(32)), ('HMAC', gen_random_bytes(32));
8
9 CREATE FUNCTION vault.enc(p_plain TEXT) RETURNS BYTEA
10 LANGUAGE sql STABLE SECURITY DEFINER SET search_path = pg_catalog, public AS $$
11   SELECT pgp_sym_encrypt(p_plain, encode(key_bytes, 'hex'),
12     'cipher-algo=aes256, s2k-mode=1')
13 FROM vault.data_key WHERE purpose = 'ENC';
14 $$;
15
16 CREATE FUNCTION vault.blind_index(p_plain TEXT) RETURNS BYTEA
17 LANGUAGE sql STABLE SECURITY DEFINER SET search_path = pg_catalog, public AS $$
18   SELECT hmac(convert_to(p_plain, 'UTF8'), key_bytes, 'sha256')
19 FROM vault.data_key WHERE purpose = 'HMAC';
20 $$;
21 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA vault FROM PUBLIC;
22 REVOKE ALL ON SCHEMA vault FROM PUBLIC;
23
24 -- The only way to create a student: validates and encrypts the PII
25 CREATE FUNCTION sis.register_student(p_roll TEXT, p_name TEXT, p_email TEXT,
26   p_dept SMALLINT, p_dob DATE, p_phone TEXT, p_aadhaar TEXT,
27   p_db_user NAME DEFAULT NULL) RETURNS INT
28 LANGUAGE plpgsql SECURITY DEFINER SET search_path = pg_catalog, sis AS $$
29 DECLARE v_id INT;
30 BEGIN
31   IF p_phone !~ '^[6-9][0-9]{9}$' THEN RAISE EXCEPTION 'Invalid phone number'; END IF;
32   IF p_aadhaar !~ '^[2-9][0-9]{11}$' THEN RAISE EXCEPTION 'Invalid Aadhaar number'; END IF;
33   INSERT INTO sis.student (roll_no, full_name, email, dept_id, dob,
34     phone_enc, aadhaar_enc, aadhaar_bidx, db_user)
35   VALUES (upper(p_roll), p_name, lower(p_email), p_dept, p_dob,
36     vault.enc(p_phone), vault.enc(p_aadhaar),
37     vault.blind_index(p_aadhaar), p_db_user)
38   RETURNING student_id INTO v_id;
39   RETURN v_id;
40 END $$;

```

Result at rest. Read directly from the table, even by a superuser, the Aadhaar column holds only ciphertext. A 12-digit number becomes 77 bytes of OpenPGP data:

roll_no	aadhaar_enc (first 12 bytes, hex)	Bytes
25881A0301	c30c0409010214de03ac3bc1...	77
25881A0401	c30c040901024bfaf03960e6...	77
25881A0402	c30c0409010257527b0c5ef4...	77

2.6.2 Break-Glass Access to Plaintext

Some staff occasionally need a full Aadhaar or phone number, for example for scholarship verification. Instead of granting column access, the registrar calls `sis.reveal_pii()`. It requires a written justification, logs the access *before* returning data, and refuses after 20 reveals per user per day. This limits mass extraction even with a stolen registrar password.

Listing 8: Audited, rate-limited PII reveal (from 05.audit.sql)

```

1 CREATE FUNCTION sis.reveal_pii(p_roll TEXT, p_reason TEXT)
2 RETURNS TABLE (roll_no TEXT, phone TEXT, aadhaar TEXT)
3 LANGUAGE plpgsql SECURITY DEFINER SET search_path = pg_catalog, sis AS $$
4 BEGIN
5     IF length(trim(COALESCE(p_reason, ''))) < 10 THEN
6         RAISE EXCEPTION 'A justification of at least 10 characters is required';
7     END IF;
8     IF (SELECT COUNT(*) FROM audit.log l
9         WHERE l.login_user = session_user AND l.action = 'REVEAL_PII'
10        AND l.logged_at >= date_trunc('day', now())) >= 20 THEN
11        RAISE EXCEPTION 'Daily PII reveal limit reached for %', session_user;
12    END IF;
13
14    INSERT INTO audit.log (action, table_name, row_key, reason)
15    VALUES ('REVEAL_PII', 'student', upper(p_roll), p_reason);
16
17    RETURN QUERY
18    SELECT s.roll_no::TEXT, vault.dec(s.phone_enc), vault.dec(s.aadhaar_enc)
19        FROM sis.student s
20        WHERE s.roll_no = upper(p_roll);
21    IF NOT FOUND THEN
22        RAISE EXCEPTION 'No student with roll number %', p_roll;
23    END IF;
24 END $$;
25 GRANT EXECUTE ON FUNCTION sis.reveal_pii(TEXT, TEXT) TO r_admin_office;

```

2.7 Audit Logging

2.7.1 Design

- A generic row-level AFTER trigger (for INSERT, UPDATE and DELETE) on `student`, `marks` and `fee_payment` writes to `audit.log`. It records the human login (`session_user`), client IP, time, table, primary key and data.
- For UPDATE only the *changed* columns are stored (old and new values), which keeps the log small. Ciphertext columns are never copied into the log.
- The trigger function is SECURITY DEFINER, so users who cause changes do not need, and do not have, any privilege on the log itself.
- **Append-only:** BEFORE UPDATE/DELETE and BEFORE TRUNCATE triggers reject every modification, even by the owner. Only `r_auditor` may read the log (*separation of duties*).

Listing 9: Generic change-capture trigger and append-only protection

```

1 CREATE FUNCTION audit.f_row_change() RETURNS TRIGGER
2 LANGUAGE plpgsql SECURITY DEFINER SET search_path = pg_catalog AS $$
3 DECLARE
4     v_hidden TEXT[] := ARRAY['phone_enc', 'aadhaar_enc', 'aadhaar_bidx'];
5     v_old JSONB; v_new JSONB; v_row JSONB; v_key TEXT;
6 BEGIN
7     IF TG_OP <> 'INSERT' THEN v_old := to_jsonb(OLD) - v_hidden; END IF;
8     IF TG_OP <> 'DELETE' THEN v_new := to_jsonb(NEW) - v_hidden; END IF;
9     v_row := COALESCE(v_new, v_old);
10    SELECT string_agg(v_row ->> col, ',') INTO v_key FROM unnest(TG_ARGV) AS col;
11
12    IF TG_OP = 'UPDATE' THEN
13        SELECT jsonb_object_agg(n.key, n.value), jsonb_object_agg(n.key, v_old -> n.key)
14            INTO v_new, v_old
15        FROM jsonb_each(v_new) n
16        WHERE n.value IS DISTINCT FROM v_old -> n.key;
17        IF v_new IS NULL THEN RETURN NULL; END IF;
18    END IF;
19
20    INSERT INTO audit.log (action, table_name, row_key, old_data, new_data)
21    VALUES (TG_OP, TG_TABLE_NAME, v_key, v_old, v_new);

```

```

22     RETURN NULL;
23 END $$;
24
25 CREATE TRIGGER trg_audit_marks AFTER INSERT OR UPDATE OR DELETE ON sis.marks
26     FOR EACH ROW EXECUTE FUNCTION audit.f_row_change('student_id', 'course_id');
27
28 CREATE FUNCTION audit.f_append_only() RETURNS TRIGGER LANGUAGE plpgsql AS $$
29 BEGIN
30     RAISE EXCEPTION 'audit.log is append-only (% blocked)', TG_OP;
31 END $$;
32 CREATE TRIGGER trg_log_no_update BEFORE UPDATE OR DELETE ON audit.log
33     FOR EACH ROW EXECUTE FUNCTION audit.f_append_only();
34 CREATE TRIGGER trg_log_no_truncate BEFORE TRUNCATE ON audit.log
35     FOR EACH STATEMENT EXECUTE FUNCTION audit.f_append_only();

```

2.7.2 What the Auditor Sees

After the verification script ran, the auditor `aud_sharma` saw exactly the three security-relevant events:

Table 8: Contents of `audit.log` as seen by the auditor

ID	Login	Action	Table	Key	Old → New	Reason
1	fac_kumar	UPDATE	marks	1,1	internal: 35 → 38	–
2	admin_ravi	REVEAL_PII	student	25881A0502	–	Scholarship verif. ...
3	admin_ravi	LOOKUP_AADHAAR	student	25881A0401	–	blind-index search

The same auditor got `permission denied for schema sis` when trying to read student data directly, so auditors can check the trail but cannot browse the records themselves.

2.8 Performance Overhead Measurement

2.8.1 Method

- **Environment:** PostgreSQL 16.14 on a 4-vCPU Intel Xeon (2.1 GHz) Linux server with 15 GB RAM; client: Python 3 + `psycpg2` on the same host.
- **Data:** 20,006 students in 5 departments (40,012 encrypted values) and 40,009 marks rows, generated by `07_bench_data.sql`.
- **Procedure:** `app/benchmark.py` runs each query as the real user (via `SET SESSION AUTHORIZATION`). Read queries are repeated 200 times after 10 warm-up runs; bulk statements 5 times. The **median** is reported, and the whole benchmark was repeated 4 times (the table shows the median of the 4 runs). All changes are rolled back after each measurement.
- **Overhead** is computed as

$$\text{Overhead (\%)} = \frac{T_{\text{secured}} - T_{\text{baseline}}}{T_{\text{baseline}}} \times 100$$

Differences below about $\pm 5\%$ are within run-to-run noise.

2.8.2 Results

Table 9: Measured cost of each security control

Control (workload)	Compared with	Baseline	Secured	Overhead
RLS, faculty lists own department (4,003 rows), indexed policy column	owner + WHERE	3.39 ms	3.30 ms	$\approx 0\%$
RLS, faculty marks sheet (8,006 rows, 3-table join)	owner + WHERE	12.07 ms	13.22 ms	+9.5%
RLS, policy column <i>not</i> indexed, (SELECT fn())	owner + WHERE	3.89 ms	4.28 ms	+10.0%
RLS, policy column <i>not</i> indexed, fn() per row	owner + WHERE	3.89 ms	42.54 ms	+992%
Encrypt 2 columns on insert, AES-256, s2k-mode=1	plaintext	0.56 μ s/row	20.97 μ s/row	+20.4 μ s/row
Encrypt 2 columns on insert, AES-256, default s2k-mode=3	plaintext	0.56 μ s/row	555.90 μ s/row	+555 μ s/row
Decrypt and mask 5,000 rows (10,000 values)	plaintext	2.20 ms	72.19 ms	+7.0 μ s/value
Audit trigger, single-row UPDATE	trigger off	0.116 ms	0.184 ms	+58%
Audit trigger, 4,003-row UPDATE	trigger off	18.15 ms	131.21 ms	+623%
Parameterized vs. concatenated query	concatenated	0.074 ms	0.076 ms	$\approx 0\%$

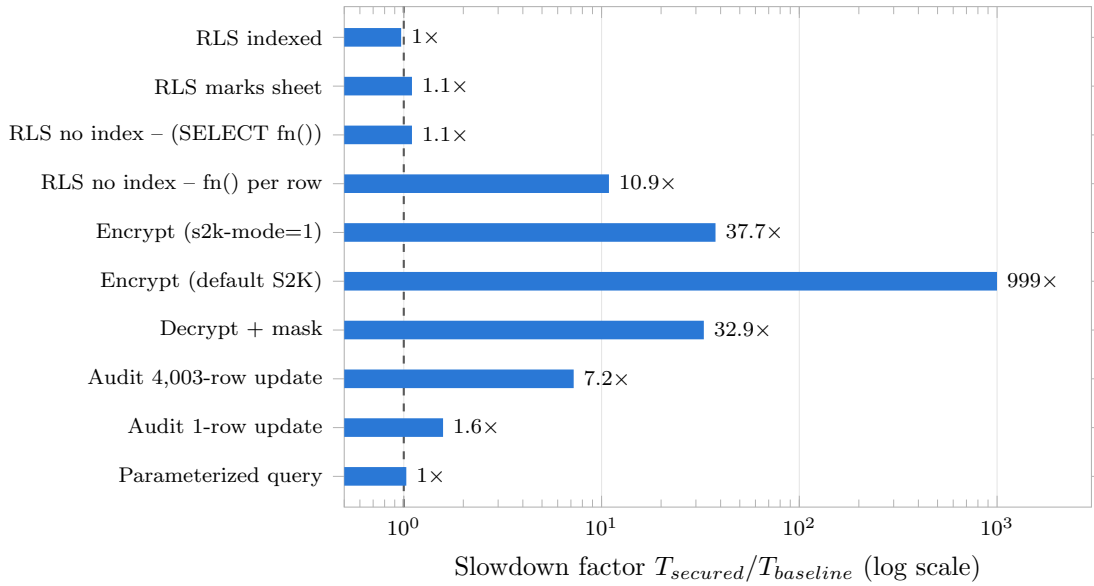


Figure 2: Relative cost of each control ($1\times$ = no overhead). Absolute costs are in Table 9.

2.8.3 Analysis and Impact on Usability

- **RLS is almost free when designed well.** With an index on the policy column and the helper wrapped as (SELECT fn()), faculty queries ran within noise of the unrestricted owner query, and the 3-table marks sheet was only 9.5% slower (about 1 ms). The *same rule* written as fn() on an unindexed column was **11 $\times$ slower**, because the function ran once for each of the 20,006 rows. Policy design, not RLS itself, decides the cost.
- **Encryption looks expensive in percent but is cheap in absolute terms.** Encrypting two columns adds about 21 μ s per registration, which no user can notice. Decryption

costs about $7\mu\text{s}$ per value: a page listing 50 masked students adds under 1 ms, while exporting all 20,000 masked records adds about 0.3 s. The real danger was the *default* key-stretching mode, which is **26 $\times$ slower** (bulk registration of 20,000 students would take about 11 s instead of 0.4 s) and adds no security for a random 256-bit key.

- **Auditing has a visible cost only for bulk operations.** A single mark entry went from 0.12 to 0.18 ms, which is imperceptible. Uploading 4,003 marks at once took 0.13 s instead of 0.02 s (about $28\mu\text{s}$ per row), still acceptable for an occasional bulk upload.
- **The SQL injection fix costs nothing.** Parameterized queries performed the same as string concatenation.

2.9 Conflict Between Security, Usability and Performance

Table 10: Trade-off analysis and the resolution adopted

Control	Security gain	Usability cost	Measured performance cost	Resolution adopted
Per-user logins + RLS	Database enforces “who sees what” even if the application is buggy	More accounts to manage; harder connection pooling	$\approx 0\text{--}10\%$	Group roles; scripted account creation; index policy columns
Column privileges + masking	Faculty and accounts never see raw PII	Accounts can verify identity only by the last 4 digits	$7\mu\text{s}$ per masked value	Masked view for routine work; break-glass reveal for exceptions
Column encryption	Stolen files and backups are useless	No LIKE, range search or sorting on PII	$+21\mu\text{s}$ per insert	HMAC blind index for exact search; <code>s2k-mode=1</code>
Break-glass reveal	Every disclosure is accountable; mass extraction blocked	Registrar must type a reason; 20 per day	Negligible	Limit is configurable; supervisors can raise it
Audit triggers	Non-repudiation and forensics	Log grows; someone must review it	$+58\%$ single row, $7\times$ bulk	Store only changed columns; statement-level triggers for bulk loads (future)
Parameterized queries	Removes SQL injection	Developer discipline needed	≈ 0	Enforced through code review

The main conclusion is that the conflict between security and performance is real, but it is concentrated in a few places (bulk audit and bulk decryption) that users rarely hit. Most of the cost found in testing came from *design choices*, such as the default key stretching and unindexed policy functions, not from the security controls themselves. Measuring each control turned an assumed conflict into a set of specific, solvable engineering decisions.

2.10 Proposed Solution Strategy

Table 11: Summary of the security architecture

Strategy	Implementation Details
Threat Modelling	STRIDE analysis of 10 threats with $L \times I$ risk scoring; controls prioritised by risk.
Access Control	Non-login owner, 5 group roles, personal logins, column-level grants, RLS policies, definer/invoker views.
Injection Defence	Allow-list validation + parameterized queries; least privilege as the second layer.
Encryption	AES-256 column encryption (<code>pgcrypto</code>), isolated key vault, HMAC blind index, TLS + SCRAM-SHA-256.
Accountability	Change-capture triggers, audited break-glass PII access, append-only log readable only by auditors.
Performance	Indexed policy columns, (<code>SELECT fn()</code>) policies, <code>s2k-mode=1</code> , diff-only audit records.

3 Sustainability Integration

3.1 Efficient and Responsible Data Handling

Security controls consume CPU time, storage and therefore energy, so they should be engineered as carefully as any other resource:

- **Computation:** Choosing `s2k-mode=1` instead of the default removes about 96% of the CPU work per encryption (20.97 vs. 555.90 μ s per row), with no loss of security for random keys. Writing RLS policies as (`SELECT fn()`) avoids an 11 $\times$ slowdown.
- **Storage:** The audit trigger stores only changed columns. A mark correction is logged as `{"internal": 38}` instead of the full row, which keeps the always-growing log small.
- **Data minimisation:** Staff see masked values by default, and plaintext is revealed only on demand. This follows the data-minimisation principle of the DPDP Act and reduces how much sensitive data is ever exposed.

3.2 Impact on Society and Environment

- **Social:** Students' identity numbers and academic records are protected against theft, misuse and tampering. This upholds their right to privacy and their trust in the institution (**SDG 16**, **SDG 4**).
- **Economic:** Preventing a breach avoids regulatory penalties, legal costs and loss of reputation. Security built into the database also avoids repeated expensive fixes in each application.
- **Environmental:** Measuring overhead and removing wasteful cryptographic and query work lowers the energy needed to run the system securely.

4 Conclusion and Future Directions

4.1 Solution Summary

A STRIDE threat model of a student information system identified ten threats, with SQL injection and mark tampering as the highest risks. The resulting design combines a non-login owner, five least-privilege group roles, column grants, row-level security and two kinds of views. Aadhaar and phone numbers are encrypted with AES-256 behind an isolated key vault, with a blind index for search, masked display and an audited, rate-limited break-glass reveal. Every change to sensitive tables is recorded in an append-only audit log that only auditors can read. An SQL injection that leaked the encryption keys and altered every mark through an over-privileged account was contained by least privilege and eliminated by parameterized queries. Measurements showed that well-designed RLS and parameterized queries are almost free. Encryption and auditing add only microseconds per row, so their cost matters only for bulk operations.

4.2 Future Scope

Future work includes moving the keys to an external **Key Management Service or Hardware Security Module** with envelope encryption and periodic key rotation. Statement-level auditing with the **pgAudit** extension, and **multi-factor authentication** for privileged users, are further steps. **Hash-chained audit records** would make tampering detectable even by a superuser. **Machine-learning anomaly detection** over the audit log could flag unusual access patterns, such as many reveals late at night. Finally, **statement-level triggers with transition tables** would reduce the bulk-audit overhead.

Bibliography

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2016 (Chapter 30: Database Security).
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019.
- [3] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” *Proceedings of the IEEE*, Vol. 63, No. 9, pp. 1278–1308, 1975.
- [4] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.
- [5] W. G. J. Halfond, J. Viegas and A. Orso, “A Classification of SQL-Injection Attacks and Countermeasures,” *Proc. IEEE International Symposium on Secure Software Engineering*, 2006.
- [6] OWASP Foundation, *SQL Injection Prevention Cheat Sheet* and *OWASP Top 10:2021 – A03 Injection*, <https://owasp.org>.
- [7] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation: Row Security Policies, CREATE VIEW, and the pgcrypto module*, <https://www.postgresql.org/docs/16/>.
- [8] J. Callas et al., “OpenPGP Message Format,” RFC 4880, IETF, 2007.
- [9] E. Barker, *Recommendation for Key Management: Part 1 – General*, NIST Special Publication 800-57 Part 1 Rev. 5, 2020.
- [10] Government of India, *The Digital Personal Data Protection Act, 2023* (No. 22 of 2023), and the Digital Personal Data Protection Rules, 2025.